# JL4 Role Goals

Sample role-expectation document for MyImpact goal import and review

| | |
|---|---|
| **Role level** | JL4 — Experienced Software Engineer / Senior Individual Contributor |
| **Purpose** | Illustrative sample for testing goal extraction, scope inference, draft creation, and duplicate detection in MyImpact. |
| **Important note** | These are generic example expectations, not an official Maersk JL4 competency framework. Replace with approved internal wording before performance evaluation. |

## How to use these goals

Each goal includes an outcome, success indicators, example evidence, and expected impact. When imported, goals should be created as drafts for the user to review and edit before approval or activation.

### JL4-G1 — Deliver reliable, maintainable software features

| | |
|---|---|
| **Scope** | PERSONAL |
| **Goal** | Design, implement, test, and deliver assigned features that meet agreed requirements, follow engineering standards, and remain maintainable after release. |

**Success indicators**

- Break requirements into small, reviewable implementation tasks and communicate trade-offs early.
- Add or update unit and integration tests for relevant behavior and edge cases.
- Address code-review feedback and align changes with agreed architecture and coding conventions.

**Example evidence:** Pull requests and review history; technical stories; automated test results; release notes; defect or rework trends.

**Expected impact:** Improves delivery confidence, reduces avoidable regressions, and makes the codebase easier to maintain and extend.

### JL4-G2 — Improve service reliability and operational readiness

| | |
|---|---|
| **Scope** | TEAM |
| **Goal** | Contribute to stable production services by identifying recurring failure patterns, improving observability, and following through on operational improvements. |

**Success indicators**

- Investigate assigned incidents using logs, metrics, traces, and documented troubleshooting steps.
- Improve dashboards, alerts, health checks, or runbooks where an operational gap exists.
- Complete agreed incident follow-ups and share useful learnings with the team.

**Example evidence:** Incident tickets and follow-up actions; dashboard/alert changes; runbook updates; service metrics; operations feedback.

**Expected impact:** Shortens diagnosis time, improves service visibility, and helps prevent repeated incidents.

### JL4-G3 — Strengthen automated testing and code quality

| | |
|---|---|
| **Scope** | PERSONAL |
| **Goal** | Increase confidence in changes by improving meaningful automated test coverage and addressing maintainability issues in owned components. |

**Success indicators**

- Identify high-risk paths and add tests for normal, boundary, and failure scenarios.

- Resolve or document important static-analysis findings and technical debt.
- Keep tests deterministic and suitable for continuous integration.

**Example evidence:** Test cases and CI results; coverage trends; static-analysis reports; resolved technical-debt items; defect escape data.

**Expected impact:** Reduces regression risk and improves the team's ability to change software safely.

## JL4-G4 — Collaborate effectively and share technical knowledge

| | |
|---|---|
| **Scope** | TEAM |
| **Goal** | Help teammates deliver work through clear communication, constructive design and code reviews, and reusable technical documentation. |

**Success indicators**

- Provide timely, actionable code-review feedback and respond constructively to feedback.
- Document implementation decisions, operational procedures, or recurring troubleshooting patterns.
- Share learnings through a walkthrough, pairing session, or team discussion when useful.

**Example evidence:** Review records; design notes; documentation links; walkthrough materials; peer feedback.

**Expected impact:** Reduces knowledge silos, supports consistent engineering practices, and helps the team resolve problems efficiently.

## JL4-G5 — Contribute to incremental technical improvement

| | |
|---|---|
| **Scope** | ORGANIZATION |
| **Goal** | Identify and deliver a bounded improvement that makes a shared engineering workflow, component, or service more efficient, secure, scalable, or maintainable. |

**Success indicators**

- Describe the problem and establish a baseline before proposing a change.
- Evaluate options with relevant stakeholders and implement an agreed improvement.
- Measure the outcome and document limitations, follow-up work, and adoption guidance.

**Example evidence:** Improvement proposal; design decision; merged change; baseline and post-change measurements; adoption notes or stakeholder feedback.

**Expected impact:** Creates a measurable improvement beyond a single task and provides reusable benefit to other engineers or services.

# Import and review notes

The five goals are intentionally distinct so the importer can be tested for goal boundaries, scope inference, and extraction of evidence and impact statements.

## Suggested import test

- Import this PDF and confirm that five draft goals are extracted.
- Review goal titles, descriptions, scopes, success indicators, evidence examples, and impact statements.
- Edit extraction errors before approving or activating goals.
- Import equivalent goals from a DOCX with a different filename/capitalization and check that duplicates are skipped.
- Check that an unrelated goal is not incorrectly classified as a duplicate.

## Scope definitions used in this sample

| | |
|---|---|
| **PERSONAL** | The individual directly owns the goal and its outcomes. |
| **TEAM** | The goal contributes to team delivery, reliability, quality, or collaboration. |
| **ORGANIZATION** | The goal provides a broader reusable benefit across teams, services, or shared engineering practices. |